



Programación 1

Iterativas en JAVA

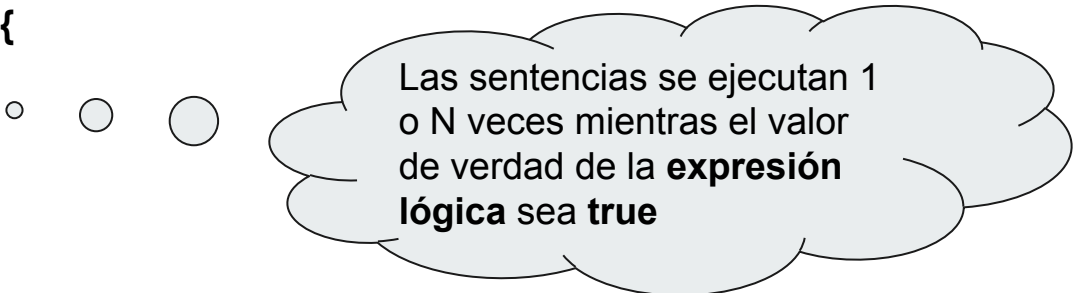
Sentencias iterativas

Un bucle o sentencia repetitiva se utiliza cuando se requiere hacer una o un conjunto de tareas varias veces

En una sentencia repetitiva hay una **expresión lógica** (condición simple o múltiple) que:

- si es cierta, ejecuta las sentencias entre llaves, y posteriormente vuelve a verificar la expresión lógica de terminación.
- si es falsa, sale del bucle.

```
Ciclo (expresión lógica) {  
    sentencia 1;  
    ...  
    sentencia n;  
}
```



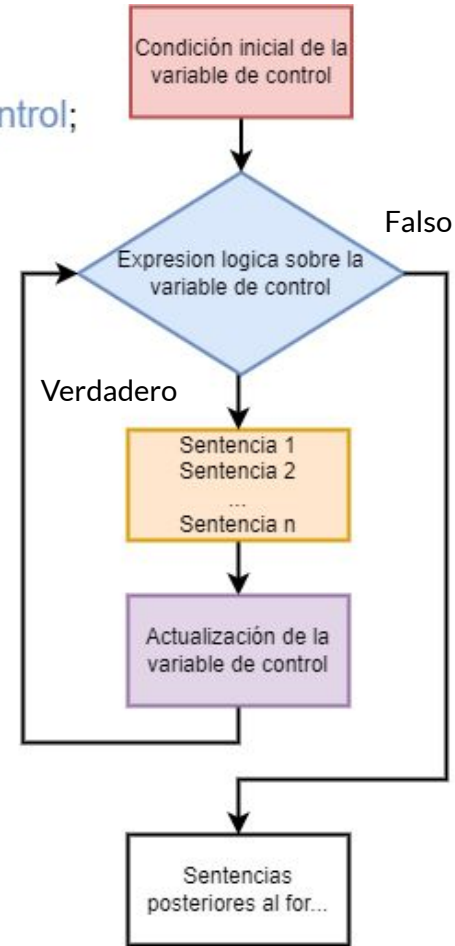
Las sentencias se ejecutan 1 o N veces mientras el valor de verdad de la **expresión lógica** sea **true**

The diagram illustrates the loop logic. It shows a code block for a loop. To the right of the code block, there are three small circles of increasing size, leading to a large, light-blue thought bubble. Inside the bubble, the text explains that the sentences are executed 1 or N times as long as the truth value of the logical expression is true.

Sentencia for

```
for( condición inicial de variable de control; expresión lógica sobre variable de control;  
    actualización de la variable de control) {  
    sentencia_1;  
    sentencia_2;  
    ...  
    sentencia_n;  
}
```

La sentencia **for** SOLO se utiliza cuando se conoce exactamente la cantidad de iteraciones.



Ejemplo



```
//Sentencia for
```

```
//Dado un número entero con valor inicial 5, imprimir la tabla de multiplicar de dicho número
```

```
public class Clase_2_Ejemplo_3 {  
    public static void main(String args[]) {  
        final int MAX = 10;  
        final int MULTIPL0 = 5;  
        System.out.println("Tabla de multiplicar del" + MULTIPL0);  
        // Se puede declarar o no la variable multiplicador dentro del inicio del for  
        for (int multiplicador = 1; multiplicador <= MAX; multiplicador++) {  
            System.out.println(MULTIPL0+" * "+multiplicador+" = "+(MULTIPL0 * multiplicador));  
        }  
    }  
}
```

Ejemplo

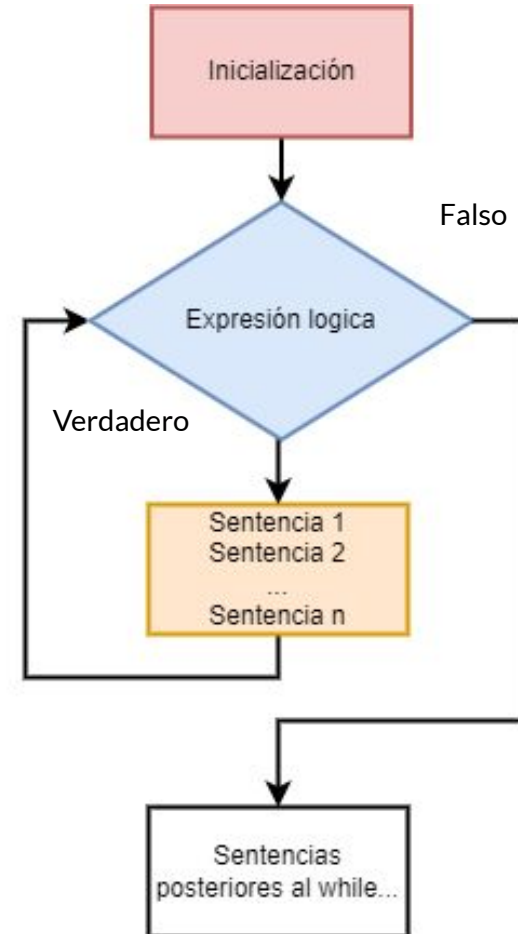
//Sentencia for anidada

```
public class Clase_2_Ejemplo_4 {  
    public static void main(String args[]) {  
        final int MAX = 10;  
        final int MULTIPL0 = 3;  
        System.out.println("Tablas de multiplicar del 1, 2 y 3");  
        for (int i = 1; i <= MULTIPL0; i++) {  
            System.out.println("Tabla de multiplicar del " + i);  
            for (int j = 1; j <= MAX; j++) {  
                System.out.println(j + " * " + i + " = " + j * i);  
            }  
        }  
    }  
}
```

Sentencia While

```
[inicialización]
while( expresión lógica ) {
    sentencia_1;
    sentencia_2;
    ...
    sentencia_n;
}
```

La sentencia while se utiliza cuando se conoce exactamente o **NO** la cantidad de iteraciones.



Sentencia While



- Una sentencia iterativa se utiliza cuando se requiere realizar una o muchas tareas varias veces (varias veces puede ser ninguna, una o más).
- En la sentencia iterativa **while (expresión lógica){...}** hay una expresión lógica que:
 - si es **verdadera**, **ejecuta** las sentencias entre llaves, y luego vuelve al **inicio** de la sentencia iterativa para evaluar nuevamente la expresión lógica.
 - si es **falsa**, **sale** de la iteración.
- Mientras la **expresión lógica** sea verdadera va a ejecutarse lo que está dentro de las {...} en el orden en que aparecen.
- Algún valor de las **variables** de la expresión lógica deberá **cambiar** en algún ciclo de la repetición dentro de las sentencias en las {...}, sino se producirá un **ciclo infinito**.

Ejemplo While

/*Dado un número entero con valor inicial 1, hacer una iteración que haga incrementar el número de a uno hasta un valor MAX = 4 (constante). Mientras itera deberá imprimir el número

*/

```
public class Clase_2_Ejemplo_5 {  
    public static void main (String [] args) {  
        final int MAX = 4;  
        int numero = 1;  
        while (numero <= MAX){  
            System.out.println("El numero es: " + numero);  
            //al cambiar de valor el número significa que el valor de la  
            //expresión lógica va a cambiar  
            numero++;  
        }  
    }  
}
```




Ejemplo: sumar números en un rango

Dado un número N mayor a 0 (ingresado por el usuario), calcular e informar por consola la suma de todos los números enteros comprendidos entre 1 y N .

Ejemplo: Para $N = 5$ el resultado es 15 dado que $1 + 2 + 3 + 4 + 5 = 15$.

Ejemplo: sumar números en un rango

```
public class Clase2_Ejemplo6 {  
    public static void main(String[] args) {  
        int numero = 0;  
        int suma = 0;  
        System.out.println("Ingrese un número mayor a 0:");  
        numero = Utils.leerInt();  
        if (numero > 0) {  
            int i = 1;  
            while (i <= numero) {  
                suma += i;  
                i++;  
            }  
            System.out.println("La suma de numeros entre 1 y " + numero + " es " + suma);  
        }  
        else {  
            System.out.println("El número ingresado NO es mayor a 0");  
        }  
    }  
}
```

Ejemplo valor positivo válido



Para este problema deberá utilizar todos los tipos de sentencias desarrolladas hasta el momento.

Ingresar un valor positivo **válido** y mostrarlo.

Nota: se debe continuar solicitando un valor hasta obtener lo pedido.

Ejemplo valor positivo válido



```
public class Clase_2_Ejemplo_7 {  
    public static void main(String args[]) {  
        int valor = 0;  
        while (valor <= 0) { // los valores positivos no incluyen al 0  
            System.out.println("Ingrese valor positivo: ");  
            valor = Utils.leerInt();  
            System.out.println("El valor ingresado fue: " + valor);  
        }  
    }  
}
```

Ejemplo: adivinar el número secreto



Almacenar en una variable un número secreto de un dígito (usar `Math.random()`). Pedirle al usuario que en 5 intentos adivine dicho número secreto. Si lo logra, informar en cuantos intentos lo hizo, sino informar el número secreto que no adivinó.

Ejemplo: adivinar el número secreto

```
public class Adivinador {  
    public static void main(String[] args) {  
        final int MAX_INTENTOS = 5;  
        int numeroSecreto = (int)(Math.random() * 10);  
        int numero = -1;  
        int cantIntentos = 0;  
  
        System.out.println("Tiene " + MAX_INTENTOS + " intentos para adivinar mi número");  
        while (cantIntentos < MAX_INTENTOS && numero != numeroSecreto) {  
            System.out.println("Ingrese un número entero:");  
            numero = Utils.leerInt();  
            cantIntentos++;  
        }  
  
        if (numero != numeroSecreto) {  
            System.out.println("No lo logró... el número era " + numeroSecreto);  
        }  
        else {  
            System.out.println("Muy bien, encontró el número secreto en " + cantIntentos + " intento/s..");  
        }  
    }  
}
```

Varias formas de hacer lo mismo. Algunas con buenas prácticas, muchas con malas en el contexto de Programación 1!

Ejemplo: adivinar el número secreto

Utilizando while con dos condiciones lógicas

```
System.out.println("Tiene " + MAX_INTENTOS + " intentos para adivinar mi número");
while (cantIntentos < MAX_INTENTOS && numero != numeroSecreto) {
    System.out.println("Ingrese un número entero:");
    numero = Utils.leerInt();
    cantIntentos++;
}
```



Utilizando while con una condición lógica, una variable booleana y una sentencia IF

```
System.out.println("Tiene " + MAX_INTENTOS + " intentos para adivinar mi número");
while (cantIntentos < MAX_INTENTOS && !loEncontre) {
    System.out.println("Ingrese un número entero:");
    numero = Utils.leerInt();
    cantIntentos++;
    if (numero == numeroSecreto)
        loEncontre = true;
}
```



[illegible]

```
System.out.println("Tiene " + MAX_INTENTOS + " intentos para adivinar mi número");
for (cantIntentos = 1; cantIntentos <= MAX_INTENTOS; cantIntentos++) { // NOO, es WHILE
    System.out.println("Ingrese un número entero:");
    numero = Utils.leerInt();
    if (numero == numeroSecreto)
        loEncontre = true;
}
```



Ejemplo: adivinar el número secreto

Utilizando for con break (NOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOO!)

```
System.out.println("Tiene " + MAX_INTENTOS + " intentos para adivinar mi número");
for (cantIntentos = 1; cantIntentos <= MAX_INTENTOS; cantIntentos++) { // NOO, es WHILE
    System.out.println("Ingrese un número entero:");
    numero = Utils.leerInt();
    if (numero == numeroSecreto)
        break; // NOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOO!
}
```



Utilizando for y modificando la variable de control (NOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOO!)

```
System.out.println("Tiene " + MAX_INTENTOS + " intentos para adivinar mi número");
for (cantIntentos = 1; cantIntentos <= MAX_INTENTOS; cantIntentos++) { // NOO, es WHILE
    System.out.println("Ingrese un número entero:");
    numero = Utils.leerInt();
    if (numero == numeroSecreto)
        cantIntentos = MAX_INTENTOS + 1; // NOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOO!
}
```



Algunos tips



- La sentencia while (expresión lógica) {...} se utiliza cuando se conoce exactamente o no la cantidad de iteraciones.
- La sentencia for (; ;) {...} se utiliza cuando se conoce exactamente la cantidad de iteraciones.
- **No modificar la variable de control del for para forzar el corte.**
- **No forzar el corte con un break ni con return sea en for o while (aprender a armar bien la expresión lógica para que corte solo).**
- Por ejemplo cuando la expresión lógica tiene una variable que se carga dentro de las {...} o su valor es resultado de un cálculo (no de un incremento o decremento) usar while (expresión lógica) {...} ya que se desconoce cuántas iteraciones hará la sentencia.